

layout, data redundancy, and so on,) and acts as an arbitrary data store from which file systems can be created. File systems are no longer constrained to individual devices, allowing them to share space with all file systems in the pool. You no longer need to predetermine the size of a file system, as file systems grow automatically within the space allocated to the storage pool. When new storage is added, all file systems within the pool can immediately use the additional space without additional work. In many ways, the storage pool acts as a virtual memory system. When a memory DIMM is added to a system, the operating system doesn't force you to invoke some commands to configure the memory and assign it to individual processes. All processes on the system automatically use the additional memory.

4.3 Transactional semantics

ZFS is a transactional file system, which means that the file system state is always consistent on disk. Traditional file systems overwrite data in place, which means that if the machine loses power, for example, between the time a data block is allocated and when it is linked into a directory, the file system will be left in an inconsistent state. Historically, this problem was solved through the use of the `fsck` command. This command was responsible for going through and verifying file system state, making an attempt to repair any inconsistencies in the process. This problem caused great pain to administrators and was never guaranteed to fix all possible problems. More recently, file systems have introduced the concept of journaling.

The journaling process records action in a separate journal, which can then be replayed safely if a system crash occurs. This process introduces unnecessary overhead, because the data needs to be written twice, and often results in a new set of problems, such as when the journal can't be replayed properly. With a transactional file system, data is managed using copy on write semantics. Data is never overwritten, and any sequence of operations is either entirely committed or entirely ignored. This mechanism means that the file system can never be corrupted through accidental loss of power or a system crash. So, no need for a `fsck` equivalent exists. While the most recently written pieces of data might be lost, the file system itself will always be consistent. In addition, synchronous data (written using the `O_DSYNC` flag) is always guaranteed to be written before returning, so it is never lost.

4.4 Checksums and self-healing data

With ZFS, all data and metadata (which carries the information about the